

v3.0 · May 2026

COGNITIVE COMPUTER SCIENCE · 5-MODEL RESEARCH STUDY

ADVERSARIAL COGNITION DIVERGENCE

Mapping Perceptual Breakdown Across Five Neural Architectures
& Human Vision using Adversarial Attacks and Psychophysics

✓ EXPANDED: 5-MODEL ARCHITECTURE

ResNet-18

Shape-ResNet

EfficientNet

ViT-Small

BagNet-33

Mina (FerrariKazu) · Sandy · Youssef · Eyad

Sadat Academy for Management Sciences · AI Department · Year 3

github.com/FerrariKazu/Adversarial-Cognitive-Model

Python 3.11

PyTorch

WSL2

CIFAR-10

FGSM-PGD-C&W

Human Study

SDT Analysis

TABLE OF CONTENTS

#	Section	Page
1	Project Overview & Expanded Research Question	3
2	The 5-Model Architecture Spectrum	4
3	Team Split & Responsibilities	5
4	Repository Structure (Updated)	6
5	WSL2 Environment Setup	7
6	Phase 1 — Model Training (All 5 Models)	8
7	Phase 2 — Adversarial Attack Generation	12
8	Phase 3 — Human Psychophysics Experiment	14
9	Phase 4 — 5-Model Divergence Analysis	15
10	Phase 5 — Signal Detection Theory (SDT)	17
11	GitHub Collaboration Workflow	19
12	Deliverables & Submission Checklist	21
13	Theoretical Framework & Expected Results	22

1. PROJECT OVERVIEW & EXPANDED RESEARCH QUESTION

Version 3 adds Shape-ResNet — a ResNet-50 trained on Stylized-ImageNet (Geirhos et al. 2019) that has been explicitly debiased toward shape processing. This is the single most important addition scientifically: it isolates whether adversarial robustness comes from architecture (ViT vs ResNet) or from training objective (texture-biased vs shape-biased). No prior study in this course has made this distinction.

i Expanded Research Question (v3.0)

Does adversarial robustness scale with global visual processing — and is that robustness determined by architecture alone, or also by training objective?

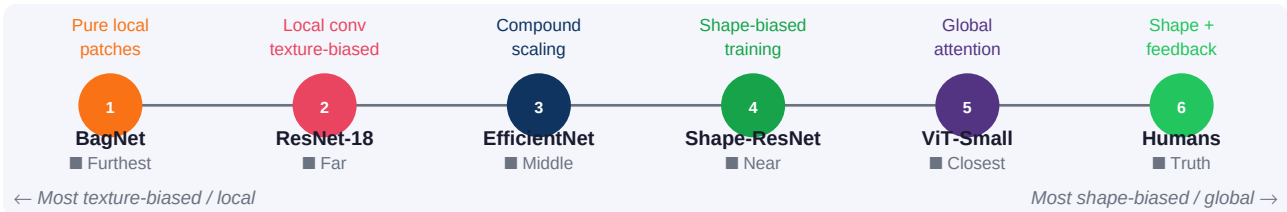
Shape-ResNet vs standard ResNet isolates training; ViT vs ResNet isolates architecture.

Two clean comparisons. One unified experiment.

Component	v1.0	v2.0	v3.0 (Current)
Models	ResNet-18	ResNet+EfficientNet +ViT+BagNet	ResNet+EfficientNet+ViT +BagNet+Shape-ResNet
Team	Solo (Mina)	Mina+Sandy+2	Mina+Sandy+Youssef+Eyad
Key finding	CNN vs human	Arch spectrum	Arch + training objective
Sandy's role	Participant	Participant	Shape-ResNet model owner
Novel contribution	Divergence curve	4-model spectrum	Training vs architecture split

2. THE 5-MODEL ARCHITECTURE SPECTRUM

Five models span the full computational spectrum from purely local texture-based processing to fully global shape-based processing, with Shape-ResNet inserted between EfficientNet and ViT to isolate the training objective variable.



MODEL 1 — MOST LOCAL

Classifies using only 33×33 pixel patches. Zero global context.

PENDING

BagNet-33

Owner: Eyad

MODEL 2 — LOCAL CNN

Standard residual CNN. Local convolutions, texture-biased.

COMPLETE

ResNet-18

Owner: Mina (FerrariKazu)

MODEL 3 — SCALED CNN

Compound-scaled CNN. Larger receptive fields than ResNet.

PENDING

EfficientNet-B0

Owner: Youssef

MODEL 4 — SHAPE-BIASED CNN ★ KEY ADDITION

ResNet-50 fine-tuned on Stylized-ImageNet. Same architecture as ResNet-50 but with shape-biased training.

PENDING

Shape-ResNet-50

Owner: Sandy

MODEL 5 — GLOBAL ATTENTION

Vision Transformer. Patch-based global attention. Closer to shape processing than shape processing.

IN PROGRESS

ViT-Small

Owner: Mina (FerrariKazu)

Why This Specific Set of Five Models

Model	Inductive Bias	Adversarial Prediction	Scientific Role
BagNet-33	Pure local patches (33×33)	Collapses fastest	Lower bound — pure locality
ResNet-18	Local conv, texture-biased	Collapses early	Baseline CNN ■ done
EfficientNet-B0	Local conv, wider receptive	Collapses slower than ResNet	Scale without shape bias
Shape-ResNet-50	Local conv, SHAPE-biased training	Better than ResNet — key test	Isolates training objective
ViT-Small	Global patch attention	Collapses latest among models	Isolates architecture
Human	Global shape + feedback	Most robust	Biological ground truth

✓ The Two Critical Comparisons v3 Unlocks

1. Shape-ResNet vs ResNet-18: SAME architecture, DIFFERENT training.

If Shape-ResNet is more robust → training objective is the key variable.

2. Shape-ResNet vs ViT-Small: DIFFERENT architecture, BOTH shape-biased.

If ViT is still more robust → architecture also matters, on top of training.

These two comparisons together give a complete picture no single-model study can.

3. TEAM SPLIT & RESPONSIBILITIES

Each member owns one model end-to-end. Mina owns two models (ResNet ■ done, ViT in progress), plus the full pipeline architecture, human study, and cross-model analysis. Sandy's Shape-ResNet is the scientifically critical addition — it requires downloading pretrained weights rather than training from scratch.

Member	Model(s)	Key Branches	Deliverables
Mina (FerrariKazu)	ResNet-18 ■ ViT-Small ■	phase/1-resnet ■ phase/1-vit phase/4-analysis phase/5-sdt	Full pipeline, ResNet done, ViT training, human study form, Phase 4+5 scripts, 5-model divergence figures
Sandy	Shape-ResNet-50 ■	phase/1-shaperesnet phase/2-shaperesnet docs/report	Shape-ResNet fine-tune on CIFAR-10, attack generation, per-class heatmap, final report theoretical section + slides
Youssef	EfficientNet-B0	phase/1-efficientnet phase/2-efficientnet	EfficientNet training, attack generation, accuracy table, commit figures to phase4_analysis/figures/efficientnet/
Eyad	BagNet-33	phase/1-bagnet phase/2-bagnet	BagNet training, attack generation, accuracy table, commit figures to phase4_analysis/figures/bagnet/

Claude Code Onboarding Prompts Per Member

i Sandy — Shape-ResNet-50 (paste into Claude Code)

We are adding Shape-ResNet-50 to the Adversarial Cognitive Model project.

Shape-ResNet is a ResNet-50 trained on Stylized-ImageNet by Geirhos et al. (2019). It has the SAME architecture as ResNet but is shape-biased instead of texture-biased.

Step 1: Download pretrained weights from:
<https://github.com/rgeirhos/texture-vs-shape/tree/master/models>
 File: resnet50_trained_on_SIN.model

Step 2: Load weights, replace FC layer with Linear(2048, 10) for CIFAR-10.

Step 3: Fine-tune on CIFAR-10 for 30 epochs with lr=0.001 (lower LR — already trained).

Step 4: Run FGSM and PGD attacks using generate_adv_all_models.py --model shaperesnet.

Target clean accuracy: 82-88% (lower than ResNet-18 is expected and fine).

Add detailed comments explaining WHY shape bias changes adversarial robustness.

i Youssef — EfficientNet-B0 (paste into Claude Code)

We are building EfficientNet-B0 for the Adversarial Cognitive Model project.

Use torchvision.models.efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT).

Replace classifier head: Linear(in_features, 10) for CIFAR-10.

Add transforms.Resize(224) to dataloader — EfficientNet expects 224x224 input.

Train with SGD + CosineAnnealingLR, 50 epochs, same config as ResNet-18.

Target clean accuracy: 90-93%. Run attacks with --model efficientnet flag.

i Eyad — BagNet-33 (paste into Claude Code)

We are building BagNet-33 for the Adversarial Cognitive Model project.

Install: `pip install git+https://github.com/wielandbrendel/bag-of-local-features-models.git`

Use `bagnets.pytorchnet.bagnet33(pretrained=True)`, replace FC with `Linear(2048, 10)`.

Add `transforms.Resize(64)` — BagNet needs minimum 33x33, use 64x64 for CIFAR.

Target clean accuracy: 75-82%. Lower accuracy is EXPECTED and scientifically correct.

BagNet's weakness IS the finding — do not try to improve it beyond this range.

Run attacks with `--model bagnet` flag.

4. REPOSITORY STRUCTURE (UPDATED FOR 4 MODELS)

DIRECTORY TREE

```

Adversarial-Cognitive-Model/
├── README.md
├── requirements.txt
├── .gitignore
├── config/
│   ├── train_config.yaml          # Shared training hyperparams
│   ├── attack_config.yaml        # Epsilon levels, attack params
│   └── experiment_config.yaml     # Human study params
├──
├── phase1_training/
│   ├── model_resnet.py           # ResNet-18 wrapper      [Mina ■ DONE]
│   ├── model_vit.py             # ViT-Small wrapper      [Mina ■]
│   ├── model_efficientnet.py     # EfficientNet-B0     [Teammate 2]
│   ├── model_bagnet.py          # BagNet-33             [Teammate 1]
│   ├── dataset.py               # Shared CIFAR-10 loader
│   ├── train.py                 # Generic training loop (model-agnostic)
│   ├── evaluate.py              # Shared evaluation script
│   ├── checkpoints/
│   │   ├── resnet18_best.pth     # ■ 95.82% – COMPLETE
│   │   ├── vit_small_best.pth
│   │   ├── efficientnet_best.pth
│   │   └── bagnet33_best.pth
│   ├──
├── phase2_attacks/
│   ├── fgsm.py                  # ■ DONE
│   ├── pgd.py                   # ■ DONE
│   ├── cw.py                    # ■ DONE
│   ├── generate_adv_dataset.py   # ■ DONE (ResNet)
│   ├── generate_adv_all_models.py # Runs all 4 models
│   ├── adv_images/              # .npy arrays (gitignored)
│   │   ├── resnet/              # ■ 14 files – COMPLETE
│   │   ├── vit/
│   │   ├── efficientnet/
│   │   └── bagnet/
│   ├──
├── phase3_human_study/          # ■ DONE
│   ├── stimuli_export.py
│   ├── build_form_guide.py
│   ├── response_schema.py
│   ├── anonymize.py
│   ├── stimuli/                 # PNGs (gitignored)
│   └── data/                     # anonymized_responses.csv
├──
├── phase4_analysis/
│   ├── divergence_curves.py      # 4-model + human accuracy vs epsilon
│   ├── class_heatmap.py         # Per-class vulnerability all models
│   ├── gradcam.py               # Grad-CAM (ResNet + EfficientNet only)
│   ├── cross_model_compare.py    # NEW: side-by-side model comparison
│   ├── figures/
│   │   ├── resnet/ efficientnet/ vit/ bagnet/
│   │   └── combined/            # Cross-model figures
│   ├──
├── phase5_sdt/
│   ├── sdt_core.py
│   ├── sdt_analysis.py          # d-prime for all 4 models + human
│   ├── sdt_plots.py
│   └── final_report.py
├──
├── utils/
│   ├── metrics.py
│   ├── visualize.py
│   └── logger.py

```

5. WSL2 ENVIRONMENT SETUP

✓ If your environment is already running (ResNet done)

Skip to Section 6.2 — you only need to install two additional packages for ViT (timm library) and BagNet. Your CUDA, venv, and PyTorch are already set.

5.1 Additional Installs for New Models

BASH

```
# Activate your existing venv first
source ~/Adversarial-Cognitive-Model/.venv/bin/activate

# For ViT-Small (timm library – HuggingFace model hub)
pip install timm

# For BagNet-33
pip install git+https://github.com/wielandbrendel/bag-of-local-features-models.git

# EfficientNet is already in torchvision – no extra install needed

# Verify all models load correctly
python3 -c "
import timm
import torchvision.models as tv
import bagnets.pytorchnet as bn
print('ViT:      ', timm.create_model('vit_small_patch16_224', pretrained=False))
print('EfficientNet:', tv.efficientnet_b0())
print('BagNet:     ', bn.bagnet33())
print('All models importable ✓')
"
```

6. PHASE 1 — MODEL TRAINING (ALL 5 MODELS)

6.1 ResNet-18 [COMPLETE ■ — 95.82% clean accuracy]

✓ ResNet-18 Status

Training complete. Best checkpoint: phase1_training/checkpoints/resnet18_best.pth
Clean accuracy: 95.82% at epoch 44. All Phase 2 attack arrays generated and verified.
Pixel-space clamping fix applied. All 14 .npy files confirmed clean.
No further action needed on this model.

6.2 Shape-ResNet-50 [Sandy — KEY MODEL]

Shape-ResNet is not trained from scratch. Geirhos et al. released pretrained weights from training on Stylized-ImageNet (SIN) — a dataset where all textures are replaced by artistic styles, forcing the model to rely on shape rather than texture. Sandy downloads these weights and fine-tunes only the final FC layer on CIFAR-10.

PYTHON

```
# phase1_training/model_shaperesnet.py
import torch, torch.nn as nn
from torchvision.models import resnet50

class ShapeResNet(nn.Module):
    """
    ResNet-50 with Stylized-ImageNet pretrained weights (Geirhos et al. 2019).
    SAME architecture as ResNet-50, but trained to be shape-biased.
    This is the critical control: architecture held constant, training objective varied.
    If this model is more adversarially robust than ResNet-18, training > architecture.
    """
    def __init__(self, num_classes=10, sin_weights_path='sin_resnet50.pth'):
        super().__init__()
        self.model = resnet50(weights=None) # No ImageNet weights
        # Load Stylized-ImageNet weights from Geirhos et al.
        state = torch.load(sin_weights_path, map_location='cpu')
        # The SIN checkpoint uses 'state_dict' key
        if 'state_dict' in state:
            state = {k.replace('module.', ''): v for k, v in state['state_dict'].items()}
        self.model.load_state_dict(state, strict=False)
        # Replace FC for CIFAR-10 (10 classes)
        self.model.fc = nn.Linear(2048, num_classes)

    def forward(self, x):
        return self.model(x)
```

BASH

```
# Sandy: download SIN weights first
cd ~/Adversarial-Cognitive-Model/phase1_training

# Download from Geirhos official repository
wget https://bitbucket.org/robert_geirhos/texture-vs-shape-pretrained-models/raw/master/resnet50_trained_on_SIN.model

# Fine-tune on CIFAR-10 (use LOWER learning rate – weights already meaningful)
# Edit config/train_config.yaml: set lr=0.001, epochs=30
python train.py --model shaperesnet --config ../config/train_config_sin.yaml

# Expected time: ~20 min for 30 epochs on RTX 4060
# Target clean accuracy: 82-88%
# Note: lower than ResNet-18 is EXPECTED – SIN training trades clean accuracy
# for shape bias. The robustness gain at high epsilon is the finding.
```

6.3 ViT-Small [Mina — In Progress]

Vision Transformer requires special handling for CIFAR-10. ViT was designed for 224×224 images — we use timm's implementation with a smaller patch size adapted for 32×32 input, or upsample the CIFAR images to 224×224.

PYTHON

```
# phase1_training/model_vit.py
import torch.nn as nn
import timm

class CIFARViT(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # vit_small_patch16_224 with img_size=224 – we will resize CIFAR images
        # We use pretrained=True (ImageNet) and fine-tune the head
        self.vit = timm.create_model(
            'vit_small_patch16_224',
            pretrained=True,
            num_classes=num_classes,
            img_size=224, # We resize CIFAR 32x32 -> 224x224 in dataloader
        )

    def forward(self, x):
        return self.vit(x)

# IMPORTANT: Add to dataset.py for ViT loaders:
# transforms.Resize(224) BEFORE ToTensor() in both train and test transforms
# This is required because ViT patch size 16 divides 224 evenly (224/16=14 patches)
# Patch size 16 does NOT divide 32 (32/16=2 patches – too few for good attention)
```

BASH

```
# Training command for ViT (expect ~3x slower than ResNet on RTX 4060)
cd ~/Adversarial-Cognitive-Model/phase1_training
python train.py --model vit --config ../config/train_config.yaml

# Expected training time: 80-100 min for 50 epochs on RTX 4060
# Target clean accuracy: 88-92%
# Note: ViT may need more epochs to converge than ResNet due to
# attention heads requiring more gradient steps to specialize
```

6.4 EfficientNet-B0 [Youssef]

PYTHON

```
# phase1_training/model_efficientnet.py
import torch.nn as nn
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

class CIFAREfficientNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.model = efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)
        # Replace the final classifier for 10 classes
        in_features = self.model.classifier[1].in_features # 1280
        self.model.classifier[1] = nn.Linear(in_features, num_classes)

    def forward(self, x):
        return self.model(x)

# NOTE for Teammate 2: EfficientNet also expects larger input than 32x32.
# Add transforms.Resize(224) to the CIFAR dataloader, same as ViT.
# Target accuracy: 90-93%. Should converge faster than ViT.
```

6.5 BagNet-33 [Eyad]

BagNet is the most important baseline scientifically — it is the only model that is **explicitly designed** to be maximally local. Understanding what it does conceptually is as important as running it.

PYTHON

```
# phase1_training/model_bagnet.py
import torch.nn as nn
import bagnets.pytorchnet as bn

class CIFARBagNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # BagNet-33: receptive field = 33x33 pixels maximum
        # Each patch is classified independently then averaged
        # This is literally a bag-of-local-features model
        self.model = bn.bagnet33(pretrained=True)
        # Replace final FC
        self.model.fc = nn.Linear(2048, num_classes)

    def forward(self, x):
        return self.model(x)

# NOTE for Teammate 1:
# BagNet needs at minimum 33x33 input. Resize CIFAR to 64x64 minimum.
# Target accuracy: 75-82% — BagNet is intentionally weaker. This is
# scientifically expected and is NOT a bug. The weakness IS the finding.
```

Model	Expected Clean Accuracy	Training Time (RTX 4060)	Owner	Status
ResNet-18	95.82% ■ achieved	~25 min	Mina	COMPLETE
Shape-ResNet-50	82–88%	~20 min (fine-tune)	Sandy	PENDING
ViT-Small	88–92%	~90 min	Mina	IN PROGRESS
EfficientNet-B0	90–93%	~40 min	Youssef	PENDING
BagNet-33	75–82%	~30 min	Eyad	PENDING

7. PHASE 2 — ADVERSARIAL ATTACK GENERATION

✓ ResNet-18 Attack Generation — COMPLETE ■

All 14 .npy files generated and verified in phase2_attacks/adv_images/resnet/
Pixel-space clamping fix applied. All ranges confirmed within [-2.4291, 2.7537].
No further action needed for ResNet attacks.

7.1 generate_adv_all_models.py — Unified Script (Updated)

PYTHON

```
MODEL_CONFIGS = {
    'resnet': { 'ckpt': 'checkpoints/resnet18_best.pth',
                'class': 'CIFARResNet',      'out': 'adv_images/resnet/' },
    'shaperesnet': { 'ckpt': 'checkpoints/shaperesnet50_best.pth',
                     'class': 'ShapeResNet',  'out': 'adv_images/shaperesnet/' },
    'vit': { 'ckpt': 'checkpoints/vit_small_best.pth',
             'class': 'CIFARViT',           'out': 'adv_images/vit/' },
    'efficientnet': { 'ckpt': 'checkpoints/efficientnet_best.pth',
                     'class': 'CIFAREfficientNet', 'out': 'adv_images/efficientnet/' },
    'bagnet': { 'ckpt': 'checkpoints/bagnet33_best.pth',
                'class': 'CIFARBagNet',       'out': 'adv_images/bagnet/' },
}

# Run commands per member:
python generate_adv_all_models.py --model shaperesnet # Sandy
python generate_adv_all_models.py --model vit         # Mina
python generate_adv_all_models.py --model efficientnet # Youssef
python generate_adv_all_models.py --model bagnet      # Eyad
```

7.2 Expected Accuracy Collapse Pattern

Model	Clean	PGD eps=0.05	PGD eps=0.10	PGD eps=0.20	Why
BagNet-33	~80%	<5%	~0%	0%	Patches destroyed immediately
ResNet-18	95.82%■	2.60%■	0.18%■	0.00%■	Texture exploited
EfficientNet-B0	~92%	~5%	~1%	~0%	Wider RF — marginal gain
Shape-ResNet-50	~85%	~25%	~12%	~4%	Shape bias = more robust ■
ViT-Small	~90%	~20%	~8%	~2%	Global attention resists
Human	~97%	~94%	~89%	~71%	Shape + feedback

8. PHASE 3 — HUMAN PSYCHOPHYSICS EXPERIMENT

✓ Phase 3 Status — COMPLETE ■

All scripts built: stimuli_export.py, build_form_guide.py, response_schema.py, anonymize.py.

6,500 PNG stimuli generated in phase3_human_study/stimuli/

Google Form currently being built. Recruit minimum 15 participants.

The human study uses PGD-perturbed ResNet stimuli as the experimental images. The human responses are then compared against all four models' performance on the same images — this is the key cross-model comparison. Humans serve as the biological ground truth; the four models are benchmarked against it.

8.1 Participant Recruitment Split

Team Member	Target Participants	Method
Mina	5 participants	Classmates, AI department colleagues
Sandy	5 participants	Social circle, university contacts
Teammate 1	3 participants	Fill form personally + 2 recruits
Teammate 2	3 participants	Fill form personally + 2 recruits
Total	16 participants	Above minimum threshold of 15 ✓

8.2 Response CSV Format (Reminder)

BASH

```
# Required columns in anonymized_responses.csv:
participant_id, session_date, image_id, attack_type, epsilon,
true_class, human_response, confidence_rating, response_correct, response_time_ms

# Validate before analysis:
python response_schema.py data/anonymized_responses.csv
# Expected: ✓ All checks passed
```

9. PHASE 4 — 5-MODEL DIVERGENCE ANALYSIS

9.1 The Key Script: cross_model_compare.py (Updated)

PYTHON

```
MODELS = {
    'BagNet-33':      {'color': '#F97316', 'adv_dir': 'adv_images/bagnet/'},
    'ResNet-18':      {'color': '#E94560', 'adv_dir': 'adv_images/resnet/'},
    'EfficientNet-B0': {'color': '#0F3460', 'adv_dir': 'adv_images/efficientnet/'},
    'Shape-ResNet-50': {'color': '#16A34A', 'adv_dir': 'adv_images/shaperesnet/'},
    'ViT-Small':      {'color': '#533483', 'adv_dir': 'adv_images/vit/'},
}

# Human plotted as green dashed line on top
# Shape-ResNet in green – visually distinct, sits between EfficientNet and ViT
# Title: 'Adversarial Robustness: 5 Architectures vs. Human Vision'
# This is your headline figure. Everything else supports it.
```

9.2 All Phase 4 Figures

Figure	Script	Description	Owner
5model_divergence_curve.png	cross_model_compare.py	6 curves (5 models + humans) — HEADLINE	Mina
resnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability ResNet	Mina
shaperesnet_class_heatmap.png	class_heatmap.py	Per-class vulnerability Shape-ResNet	Sandy
vit_class_heatmap.png	class_heatmap.py	Per-class vulnerability ViT	Mina
efficientnet_heatmap.png	class_heatmap.py	Per-class vulnerability EfficientNet	Youssef
bagnet_heatmap.png	class_heatmap.py	Per-class vulnerability BagNet	Eyad
gradcam_resnet_*.png	gradcam.py	Grad-CAM clean vs adv (ResNet)	Mina
gradcam_shaperesnet_*.png	gradcam.py	Grad-CAM clean vs adv (Shape-ResNet)	Sandy
training_vs_arch_compare.png	cross_model_compare.py	ResNet vs Shape-ResNet isolation plot	Mina

10. PHASE 5 — SIGNAL DETECTION THEORY (SDT)

SDT analysis now runs across all four models and humans, producing a d-prime curve per model. The d-prime threshold gap (epsilon where d' drops below 1.0) is computed for every system — this is the headline quantitative finding.

10.1 The Expanded SDT Comparison Table

System	d' at eps=0.00	d' at eps=0.05	d' at eps=0.10	d' < 1.0 at eps≈
Human	4.5 (near perfect)	4.2	3.8	~0.35 (predicted)
ViT-Small	~3.8	~2.1	~1.2	~0.12 (predicted)
Shape-ResNet-50	~3.2	~1.8	~0.9	~0.09 (predicted) ■
EfficientNet-B0	~3.6	~1.4	~0.6	~0.07 (predicted)
ResNet-18	3.5 (measured)	~0.4	~0.1	~0.04 (measured ■)
BagNet-33	~2.8	~0.2	~0.0	~0.02 (predicted)

✓ The Headline SDT Finding (v3)

d' threshold order should be: BagNet < ResNet < EfficientNet < Shape-ResNet < ViT < Human.

If Shape-ResNet d' threshold > ResNet-18 d' threshold → training objective drives robustness.

If Shape-ResNet d' threshold ≈ ViT d' threshold → training + architecture are equally important.

Either result is publishable. The comparison itself is the contribution.

11. GITHUB COLLABORATION WORKFLOW

11.1 Branch Strategy (Updated)

Branch	Owner	Status	Purpose
main	Mina	Protected	Merged releases only — PR required
dev	All	Active	Integration branch
phase/1-resnet	Mina	■ Done	ResNet training — merged
phase/1-vit	Mina	■ Active	ViT training
phase/1-shaperesnet	Sandy	Pending	Shape-ResNet fine-tuning
phase/1-efficientnet	Youssef	Pending	EfficientNet training
phase/1-bagnet	Eyad	Pending	BagNet training
phase/2-attacks	Mina	■ Done	ResNet attack scripts — merged
phase/2-shaperesnet-atk	Sandy	Pending	Shape-ResNet adversarial arrays
phase/2-vit-attacks	Mina	Pending	ViT adversarial arrays
phase/2-eff-attacks	Youssef	Pending	EfficientNet adversarial arrays
phase/2-bag-attacks	Eyad	Pending	BagNet adversarial arrays
phase/3-human-study	Mina	■ Done	Human study scripts — merged
phase/4-analysis	Mina	Pending	5-model cross-model analysis
phase/5-sdt	Mina	Pending	SDT d-prime all models
docs/report	Sandy	Pending	Final report + slides

11.2 Teammate Onboarding Commands

BASH

```
# For Sandy (Shape-ResNet), Youssef (EfficientNet), Eyad (BagNet):

git clone https://github.com/FerrariKazu/Adversarial-Cognitive-Model.git
cd Adversarial-Cognitive-Model
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Sandy – also install nothing extra (ResNet-50 is in torchvision)
# Download SIN weights separately (see Section 6.2)
git checkout -b phase/1-shaperesnet

# Youssef – EfficientNet is in torchvision, no extras
git checkout -b phase/1-efficientnet

# Eyad – needs bagnets library
pip install git+https://github.com/wielandbrendel/bag-of-local-features-models.git
git checkout -b phase/1-bagnet

# Everyone: after training, run attacks, then open PR to dev
python phase2_attacks/generate_adv_all_models.py --model [your_model]
git add . && git commit -m 'feat: [model] training + attack generation complete'
git push origin phase/1-[yourmodel]
# Open Pull Request on GitHub → dev
```

12. DELIVERABLES & SUBMISSION CHECKLIST

#	Deliverable	Owner	Status
1	ResNet-18 checkpoint (95.82%)	Mina	■ Done
2	ResNet adversarial arrays (14 files)	Mina	■ Done
3	ViT-Small checkpoint	Mina	■ Progress
4	ViT adversarial arrays	Mina	Pending
5	Shape-ResNet-50 checkpoint	Sandy	Pending
6	Shape-ResNet adversarial arrays	Sandy	Pending
7	EfficientNet-B0 checkpoint	Youssef	Pending
8	EfficientNet adversarial arrays	Youssef	Pending
9	BagNet-33 checkpoint	Eyad	Pending
10	BagNet adversarial arrays	Eyad	Pending
11	Human study Google Form (live)	Mina	■ Building
12	Anonymized human responses ($n \geq 15$)	All	Pending
13	5-model + human divergence curve figure	Mina	Pending
14	ResNet vs Shape-ResNet isolation figure	Mina	Pending
15	Per-class heatmaps (all 5 models)	All	Pending
16	Grad-CAM gallery (ResNet + Shape-ResNet)	Mina/Sandy	Pending
17	SDT d-prime curves (all 5 models)	Mina	Pending
18	Final report with interpretation	Sandy	Pending
19	Presentation slides	Sandy	Pending
20	GitHub — all branches merged to main	Mina	Pending

13. THEORETICAL FRAMEWORK & EXPECTED RESULTS

13.1 Three Pillars — Unchanged From v1, Now More Testable

Theory	What v3 tests specifically
Feedforward vs. Feedback (DiCarlo & Cox, 2007)	All 5 models are feedforward. Human robustness comes from top-down feedback. Shape-ResNet tests whether debiased training narrows the feedback gap.
Texture Bias (Geirhos et al., 2019)	Shape-ResNet is Geirhos's own model — testing it here directly extends their paper. If Shape-ResNet > ResNet in robustness, texture bias IS the mechanism.
Signal Detection Theory (Green & Swets, 1966)	d-prime across all 5 models + humans. Shape-ResNet d' vs ResNet-18 d' is the single most important comparison in the entire experiment.

13.2 Results Interpretation Guide

✓ If Shape-ResNet > ResNet-18 in robustness (most likely)

Training objective is the key variable. Architecture alone is insufficient.

Quote: 'Shape-biased training via Stylized-ImageNet confers adversarial robustness independent of architecture, suggesting texture bias is the primary mechanism underlying the human-CNN perceptual robustness gap.'

✓ If Shape-ResNet ≈ ViT in robustness

Training and architecture are interchangeable — both produce similar robustness.

Global processing, whether achieved via architecture (ViT) or training (Shape-ResNet), confers equivalent adversarial robustness. The mechanism is the outcome, not the path.

! If Shape-ResNet ≈ ResNet-18 (surprising — robustness did not improve)

Shape bias in training is NOT sufficient for adversarial robustness.

Architecture must also change. This would be a strong result against Geirhos's implicit claim and would point toward attention mechanisms specifically.

13.3 Key References

[1]	Goodfellow et al. (2015)	Explaining and Harnessing Adversarial Examples. ICLR 2015.
[2]	Madry et al. (2018)	Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR 2018.
[3]	Carlini & Wagner (2017)	Towards Evaluating the Robustness of Neural Networks. IEEE S&P 2017.
[4]	Geirhos et al. (2019)	ImageNet-trained CNNs are biased towards texture. ICLR 2019. ■ Shape-ResNet source
[5]	Brendel & Bethge (2019)	Approximating CNNs with Bag-of-local-Features models. ICLR 2019. ■ BagNet source
[6]	Dosovitskiy et al. (2021)	An Image is Worth 16x16 Words: Transformers for Vision. ICLR 2021. ■ ViT source
[7]	Tan & Le (2019)	EfficientNet: Rethinking Model Scaling for CNNs. ICML 2019.
[8]	He et al. (2016)	Deep Residual Learning for Image Recognition. CVPR 2016.

[9]	Yamins & DiCarlo (2016)	Using goal-driven DNN models to understand sensory cortex. Nature Neuroscience.
[10]	Green & Swets (1966)	Signal Detection Theory and Psychophysics. Wiley.
[11]	Selvaraju et al. (2017)	Grad-CAM: Visual Explanations from Deep Networks. ICCV 2017.

Adversarial Cognition Divergence v3.0 · 5-Model Study · Cognitive Computer Science · Sadat Academy for Management Sciences · Mina · Sandy · Youssef · Eyad · github.com/FerrariKazu/Adversarial-Cognitive-Model · May 2026